Git for Working in a Team — Practical Tutorial

Read top-to-bottom once; then use as a reference. GitLab-flavoured (Merge Requests), but applies to GitHub too.

0. The mental model (everything rests on this)

Git has **three areas** + your **remote**:

```
working dir   ──git add──▶  staging (index)  ──git commit──▶  local repo (.git)  ──git push──▶  remote (GitLab)
 (your files)              (what's queued)     (history of commits)                  (shared origin)
```

- A **commit** = a snapshot + message + parent(s). History is a chain of commits.
- A **branch** = a movable pointer to a commit (cheap — just a label).
- `origin` = the default name for the shared remote (GitLab).
- `HEAD` = "where you are now" (usually the tip of your current branch).

The golden rule of teams: you almost never commit to `main` directly. You work on a **branch**, push it, and open a **Merge Request (MR)** for review.

1. One-time setup

```
git config --global user.name  "Afam I-Ezike"
git config --global user.email "you@company.com"
git config --global init.defaultBranch main
git config --global pull.ff only            # SAFE default: refuse surprise merge commits;
                                            #   you then choose explicitly: git pull --rebase
# (teams wanting always-linear history use `pull.rebase true` instead — pick one, be consistent)

# SSH key so you don't type passwords (then add the public key in GitLab → Settings → SSH Keys)
ssh-keygen -t ed25519 -C "you@company.com"   # press enter through prompts
cat ~/.ssh/id_ed25519.pub                    # copy this into GitLab

git clone git@gitlab.com:team/project.git    # get the repo
cd project
```

2. The daily loop (memorise this rhythm)

```
git checkout main          # start from main
git pull                   # get everyone's latest
git checkout -b feature/add-cds-pricer   # branch for YOUR task (descriptive name)

# ... edit files ...
git status                 # what changed?
git diff                   # see the actual changes
git add path/to/file.py    # stage what you want (or: git add -p  for chunk-by-chunk)
git commit -m "Add CDS pricer with hazard-rate survival curve"

git push -u origin feature/add-cds-pricer   # first push sets upstream; later just: git push
```

Then open a **Merge Request** in GitLab (it usually prints a link after push). Repeat: edit → add → commit → push.

Commit often, in small logical units. Each commit should do one thing and build/pass tests.

3. Branching & the team model

- `main` (or `master`) = always deployable / protected. No direct commits.
- **Feature branches** = one per task, branched off `main`, short-lived, merged back via MR.
- Naming: `feature/...`, `bugfix/...`, `hotfix/...` (or `<ticket>/short-desc`, e.g. `JIRA-123/cds-pricer`).

```
git checkout main && git pull
git checkout -b feature/var-engine-vectorise   # create + switch
git switch main                                # switch back (modern alias for checkout)
git branch                                     # list local branches (* = current)
git branch -d feature/old                       # delete a merged branch locally
```

Keep branches short-lived (days, not weeks) — long branches drift and cause painful conflicts.

4. Merge Requests (the heart of team work)

An **MR** (GitLab) / **PR** (GitHub) = "please review my branch and merge it into `main`."

Workflow: 1. Push your branch → open MR (`Source: feature/... → Target: main`). 2. Write a clear **title + description** (what & why; link the ticket). 3. CI **pipeline** runs (tests/lint) — must be green. 4. Reviewers comment; you push fixes (the MR updates automatically — same branch). 5. Get **approval(s)** →

Merge. 6. Tick **"Delete source branch"** and **"Squash commits"** (keeps `main` history clean).

Respond to review by pushing more commits to the same branch:

```
# address comments...
git add . && git commit -m "Address review: bound surrogate error, add test"
git push                # MR updates — no new MR needed
```

5. Staying in sync: `fetch`, `pull`, merge vs rebase

- `git fetch` = download remote changes, *don't* touch your files (safe, look first).
- `git pull` = fetch **+** integrate into your branch.

While you work, `main` moves on. Bring those changes into your branch so your MR merges cleanly:

Rebase (preferred for feature branches — linear history):

```
git fetch origin
git rebase origin/main          # replay YOUR commits on top of the latest main
# fix any conflicts (§6), then:
git rebase --continue
git push --force-with-lease      # rebase rewrites history → force-push YOUR branch (safely)
```

Merge (alternative — preserves exact history, adds a merge commit):

```
git fetch origin
git merge origin/main
```

> **Rule of thumb: rebase your own feature branch** to stay current (clean history); **never rebase shared/ `main`** history that others have based work on. `--force-with-lease` (not plain `--force`) refuses to clobber someone else's push.

6. Resolving merge conflicts (don't panic — it's routine)

A conflict = the same lines changed two ways. Git pauses and marks them:

```
<<<<<<< HEAD
your version
=======
their version
>>>>>>> origin/main
```

Steps:

```
git status                # lists conflicted files
# open each file, edit to the correct final result, REMOVE the <<< === >>> markers
git add resolved_file.py   # mark it resolved
git rebase --continue      # (or: git merge --continue / git commit, depending on the op)
```

- Pick one whole side: `git checkout --ours file` or `git checkout --theirs file`, then `git add file`.
- ⚠️ **Canonical gotcha:** the meaning **inverts between merge and rebase.** In a **merge**, `--ours` = your current branch, `--theirs` = the incoming branch. In a **rebase**, it's **reversed** — `--ours` = the branch you're rebasing *onto* (e.g. `main`), `--theirs` = *your* commit being replayed. When unsure, just edit the file by hand rather than guess.
- A good IDE / `git mergetool` shows a 3-way view. **Re-run the tests after resolving.**
- Bail out if it's a mess: `git rebase --abort` (or `git merge --abort`) → back to before.

7. The undo toolbox (know these — they save you)

```
git stash                     # park uncommitted changes (clean tree); git stash pop to restore
git restore file.py           # discard unstaged changes to a file (was: git checkout -- file)
git restore --staged file.py  # unstage (keep the edit)

git commit --amend            # fix the LAST commit (message or add a forgotten file) — only if unpushed
git reset --soft HEAD~1       # undo last commit, KEEP changes staged
git reset --hard HEAD~1       # undo last commit, DISCARD changes  ⚠️ destructive
git revert <commit>           # make a NEW commit that undoes an old one (safe on shared history)

git reflog                    # your safety net — log of everywhere HEAD has been; recover "lost" commits
git checkout <commit-or-branch> -- file.py   # restore one file from elsewhere
```

> **Golden rule:** `revert` on anything already pushed/shared; `reset` / `amend` only on local, unpushed commits. If you think you lost work, check `git reflog` before despairing.

8. Commit hygiene & `.gitignore`

Good commit messages — **imperative mood**, ≤50-char subject, blank line, then *why* (not just *what*):

```
Add CDS pricer with hazard-rate survival curve

- protection leg via Q(t)=exp(-λt); premium leg = risky annuity
- λ from the credit triangle s/(1-R); validated vs reference (0.8816)
```

Many teams use **Conventional Commits** (canonical, machine-readable, drives changelogs/versioning):

```
feat: add CDS pricer        fix: correct swaption receiver annuity
refactor: vectorise VaR     test: add NNEG put case      docs: update git guide
chore: bump numpy           perf: cache base discount factors
```

Subject = `type(scope): summary`. One logical change per commit (**atomic commits**) — easy to review, revert, and `git bisect`. **`.gitignore`** — never commit secrets, data, build artefacts, venvs:

```
.venv/
__pycache__/
*.pyc
.env                # secrets/credentials — NEVER commit
.DS_Store
*.parquet           # large data
.ipynb_checkpoints/
```

- **Never commit credentials/keys.** If you do: rotate the secret immediately (history is forever).
- Don't commit huge binaries/data — use storage/LFS.

9. GitLab specifics

- **Protected branches:** `main` can't be pushed to directly — only via MR. (That's by design.)
- **CI/CD pipelines:** `.gitlab-ci.yml` defines jobs (lint, test, build) that run on every push/MR. **Your MR won't merge until the pipeline is green.**
- **Approvals:** projects often require N approvals before merge.
- **Draft MRs:** prefix title with `Draft:` to open early for feedback without it being mergeable.
- **Issues** link to MRs (`Closes #42` in the MR description auto-closes the issue on merge).
- **Squash & merge:** collapses your branch's commits into one clean commit on `main`.

10. Team etiquette & best practices

- **Pull `main` before branching**; rebase your branch regularly so the MR is clean.
- **Small MRs** (a few hundred lines) get reviewed fast and merge safely. Huge MRs stall.
- **One concern per MR / per commit.** Don't mix a refactor with a feature.
- **Green pipeline before asking for review.** Don't make reviewers find broken tests.
- **Review courteously and promptly**; respond to every comment (fix or explain).
- **Never force-push a shared branch** (`main`, or a branch someone else is on). `--force-with-lease` only on *your* feature branch.
- **Don't commit generated files, secrets, or data.**
- **Communicate** in the MR — the description and comments are the team's record.
- **Prune dead branches:** `git fetch --prune` drops remote-tracking refs for branches deleted on the server; delete merged local branches (`git branch -d`).
- **Atomic, reviewable commits**; rewrite history (rebase/amend) **only on your own un-pushed/feature branch**, never on `main`.

11. Common scenarios (recipes)

Situation	Do this
Start a new task	`git checkout main && git pull && git checkout -b feature/x`
"main moved, update my branch"	`git fetch && git rebase origin/main` (then `push --force-with-lease`)
Made commits on `main` by mistake	`git branch feature/x` (save them) → `git reset --hard origin/main` on main
Forgot a file in the last commit	edit → `git add file` → `git commit --amend --no-edit` (if unpushed)
Need to switch tasks, work unfinished	`git stash` → switch → later `git stash pop`
Pushed a bad commit to main (shared)	`git revert <sha>` (never reset shared history)
"What changed and who?"	`git log --oneline --graph`; `git blame file.py`; `git show <sha>`
Throw away ALL local changes	`git restore .` (unstaged) / `git reset --hard` ⚠️
Recover a "lost" commit	`git reflog` → `git checkout <sha>`

12. Command quick-reference

```
SETUP     git clone URL · git config --global user.name/email · ssh-keygen
LOOK      git status · git diff [--staged] · git log --oneline --graph · git branch
BRANCH    git checkout -b name · git switch name · git branch -d name
SAVE      git add [-p] · git commit -m "..." · git commit --amend
SYNC      git fetch --prune · git pull · git push [-u origin name] · git push --force-with-lease
INTEGRATE git rebase origin/main · git merge origin/main · (resolve) git add → --continue
UNDO      git restore [--staged] · git stash [pop] · git reset --soft/--hard · git revert · git reflog
INSPECT   git show SHA · git blame file · git diff main..feature
```

Daily mantra: **pull main → branch → small commits → push → MR → keep rebased → green pipeline → review → squash-merge → delete branch.** Do that and you'll work like a seasoned team member from day one.